

使用 Cloud Tasks 緩衝處理 HTTP 要求

來源：<https://codelabs.developers.google.com/codelabs/cloud-easy-http-cloud-tasks?hl=zh-tw>

步驟數：10

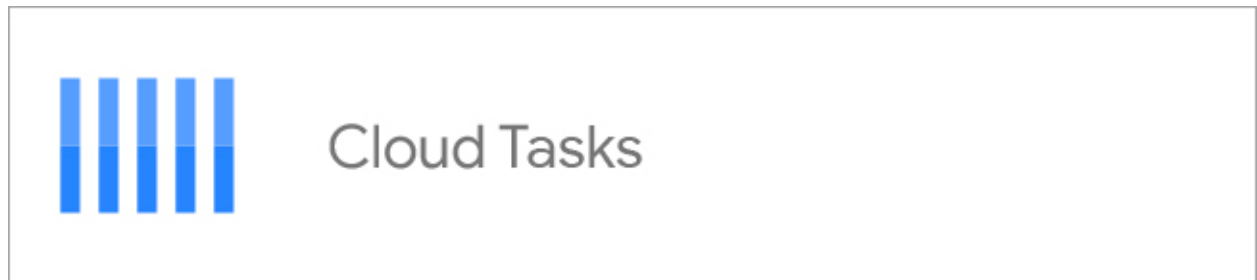
在本程式碼研究室中，您要先瞭解如何建立及使用一般 Cloud Tasks 佇列來處理 HTTP 目標工作。接著，您將會瞭解如何使用佇列層級 HTTP URI 覆寫功能和新的 BufferTask API，透過 Cloud Tasks 更輕鬆地緩衝 HTTP 要求。

目錄

1. [簡介](#)
2. [設定和需求](#)
3. [為 HTTP 目標工作建立一般佇列](#)
4. [建立及測試 HTTP 工作](#)
5. [建立具有轉送設定的佇列](#)
6. [為佇列建立及測試 HTTP 工作，並設定轉送功能](#)
7. [使用轉送設定變更待處理工作](#)
8. [為 BufferTask API 建立佇列](#)
9. [使用 BufferTask API 緩衝處理 HTTP 要求](#)
10. [恭喜](#)

1. 簡介

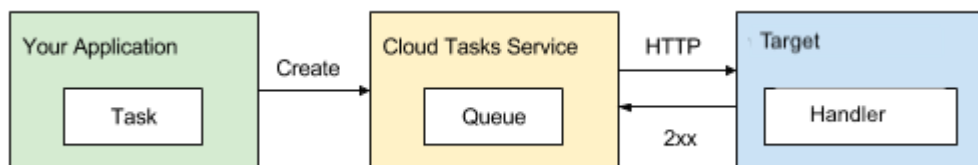
佇列層級工作轉送設定和 **BufferTaskAPI** 目前為公開預先發布版功能。這兩項功能均受《Google Cloud 服務條款》的《[正式發布前產品條款](#)》規範。正式發布前的功能只提供部分支援，對正式發布前的功能所做的變更可能與其他正式發布前的版本不相容。詳情請參閱「[推出階段說明](#)」。



Cloud Tasks 是全代管的佇列服務，可管理大量工作的執行、調度和傳送作業。

Cloud Tasks 可讓您將作業 (稱為「工作」) 分開，這些作業可在主要應用程式流程之外獨立執行 (例如更新資料庫項目的工作)，並使用您建立的處理常式以非同步方式傳送及處理。

卸載的工作會新增至**佇列**，並保留工作，直到成功執行或失敗為止。根據設定，佇列也可以做為調度流程控制項。您建立及設定佇列，然後由 Cloud Tasks 服務管理。工作新增完畢後，佇列會指派工作，並確保工作站能可靠地處理工作。



Cloud Tasks 的主要功能包括：

- **HTTP 目標**：運用業界標準 OAuth/OIDC 驗證安全地新增工作，並以 Compute Engine、Google Kubernetes Engine、Cloud Run、Cloud Functions 或內部部署系統中運作的任何 HTTP 服務為目標。
- **工作重複資料刪除**：重複新增的工作只會調度一次。
- **保證遞送**：系統保證至少遞送工作一次，而多數工作正好僅需遞送一次。
- **速率和重試控制項**：您可以設定調度工作的頻率、嘗試次數上限、在下次嘗試之前須至少等待多長時間，藉此控管執行作業。
- **未來排程**：控管工作的執行時間。

在本程式碼研究室中，您將先瞭解如何建立及使用一般 Cloud Tasks 佇列，處理 HTTP 目標工作。接著，您將瞭解如何使用佇列層級的 HTTP URI 覆寫和新的 BufferTask API，更輕鬆地透過 Cloud Tasks 緩衝處理 HTTP 要求。

課程內容

- 如何建立 HTTP 目標工作。
- 如何使用新的佇列層級 HTTP URI 覆寫功能，建立 HTTP 目標工作。
- 如何使用新的佇列層級 HTTP URI 覆寫功能，變更待處理工作。

- 如何使用新的 BufferTask API 更輕鬆地緩衝處理 HTTP 要求。
-

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' dropdown menu, which is circled in blue. Below this, the 'Select a project' section includes a search bar and a 'NEW PROJECT' button, which is highlighted with a blue squiggle. The 'New Project' form is shown below, with fields for 'Project name' (my-kubernetes-codelab) and 'Project ID' (my-kubernetes-codelab). The Project ID field is circled in blue. There's also a 'Location' field set to 'No organization' with a 'BROWSE' button. At the bottom, there are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

用者可參加[價值\\$300 美元的免費試用計畫](#)。

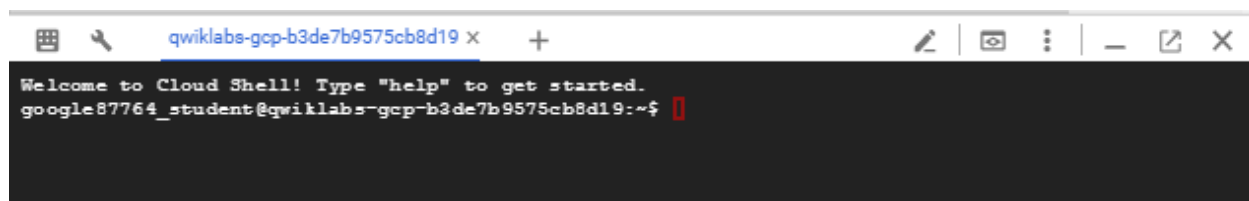
啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 [Google Cloud 控制台](#) 中，點選右上工具列的 Cloud Shell 圖示：



佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：

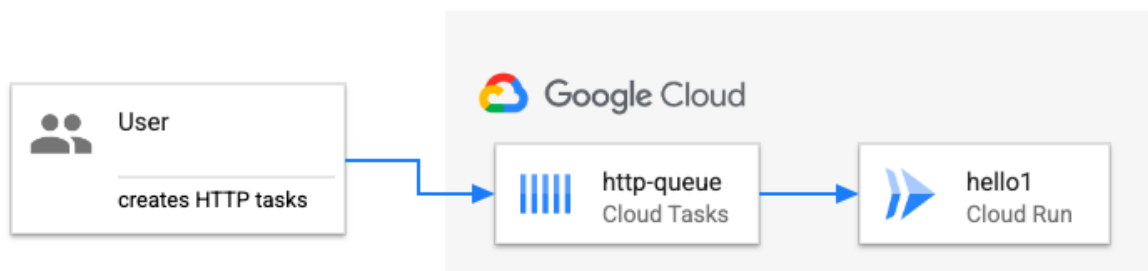
A screenshot of a web browser window showing the Google Cloud Shell terminal. The browser's address bar displays 'qwiklabs-gcp-b3de7b9575cb8d19'. The terminal window has a dark background and shows the following text: 'Welcome to Cloud Shell! Type "help" to get started.' followed by the prompt 'google87764_student@qwiklabs-gcp-b3de7b9575cb8d19:~\$' and a red cursor. The terminal window includes standard browser controls like back, forward, and close buttons.

這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您可以在瀏覽器中完成本程式碼研究室的所有作業。您不需要安裝任何軟體。

步驟 3 · 約 4 分鐘

3. 為 HTTP 目標工作建立一般佇列

在第一個步驟中，您將瞭解如何建立一般 Cloud Tasks 佇列，並在其中新增 HTTP 工作，以指定 Cloud Run 服務為目標。



什麼是 HTTP 目標工作？

HTTP 目標工作可運用業界標準 OAuth/OIDC 驗證，安全地以 Compute Engine、Google Kubernetes Engine、Cloud Run、Cloud Functions 或內部部署系統中執行的任何 HTTP 服務為目標。

部署 Cloud Run 服務

首先，請確認已啟用必要的 API：

```
gcloud services enable \
  cloudtasks.googleapis.com \
  run.googleapis.com
```

部署 Cloud Run 服務，做為 HTTP 工作的目標：

```
SERVICE1=hello1
REGION=us-central1

gcloud run deploy $SERVICE1 \
  --allow-unauthenticated \
  --image=gcr.io/cloudrun/hello \
  --region=$REGION
```

建立 Cloud Tasks 佇列

建立一般 Cloud Tasks 佇列：

```
QUEUE1=http-queue
LOCATION=us-central1

gcloud tasks queues create $QUEUE1 --location=$LOCATION
```

暫時暫停佇列，以便觀察 HTTP 工作的建立過程：

```
gcloud tasks queues pause $QUEUE1 --location=$LOCATION
```

步驟 4 · 約 2 分鐘

4. 建立及測試 HTTP 工作

在這個步驟中，您將建立 HTTP 工作，以先前建立的佇列為目標。

建立 HTTP 工作

您可以使用 `gcloud` 建立 HTTP 工作：

```
gcloud tasks create-http-task \  
  --queue=$QUEUE1 \  
  --location=$LOCATION \  
  --url=$SERVICE1_URL \  
  --method=GET
```

選用：您也可以使用用戶端程式庫建立 HTTP 工作。舉例來說，您可以查看 C# 範例的 [Program.cs](#)，瞭解如何將 HTTP 要求包裝成 `Task` 和 `TaskRequest`，然後透過 `CloudTasksClient` 傳送至 Cloud Tasks：

```
var taskRequest = new CreateTaskRequest  
{  
    Parent = new QueueName(projectId, location, queue).ToString(),  
    Task = new Task  
    {  
        HttpRequest = new HttpRequest  
        {  
            HttpMethod = HttpMethod.Get,  
            Url = url  
        }  
    }  
};  
  
var client = CloudTasksClient.Create();  
var response = client.CreateTask(taskRequest);
```

您可以執行下列指令，建立工作並將其新增至佇列：

```
dotnet run $PROJECT_ID $LOCATION $QUEUE1 $SERVICE1_URL
```

測試 HTTP 工作

此時，工作已建立但尚未執行，因為佇列已暫停。您可以列出佇列來驗證這點：

```
gcloud tasks queues list --location=$LOCATION
```

您應該會看到佇列處於 `PAUSED` 狀態：

QUEUE_NAME	STATE
http-queue	PAUSED

繼續處理佇列：

```
gcloud tasks queues resume $QUEUE --location=$LOCATION
```

查看 Cloud Run 服務的記錄：

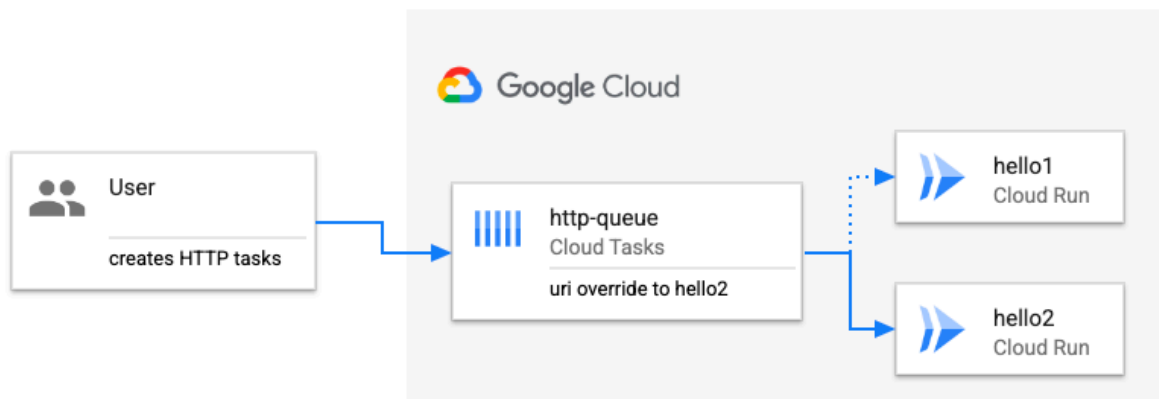
```
gcloud logging read "resource.type=cloud_run_revision AND  
resource.labels.service_name=$SERVICE1" --limit 1
```

您應該會看到 Cloud Run 服務收到來自 Cloud Tasks 的 HTTP GET 要求：

```
httpRequest:  
  latency: 0.227597158s  
  protocol: HTTP/1.1  
  remoteIp: 35.243.23.192  
  requestMethod: GET  
  requestSize: '415'  
  requestUrl: https://hello1-idcwffc3yq-uc.a.run.app/  
  responseSize: '5510'  
  serverIp: 216.239.32.53  
  status: 200  
  userAgent: Google-Cloud-Tasks
```

5. 建立具有轉送設定的佇列

在本步驟中，您將瞭解如何使用「佇列層級任務路由設定」功能，建立含有路由設定的 Cloud Tasks 佇列，並新增 HTTP URI 覆寫。然後將 HTTP 工作新增至該佇列，以第一個 Cloud Run 服務為目標，並觀察路由設定如何覆寫 URI，將工作路由至第二個 Cloud Run 服務。



什麼是佇列層級工作轉送設定？

佇列層級的工作轉送設定會變更整個佇列的 HTTP 工作轉送，適用於所有待處理和新工作。這樣一來，您就不必在工作層級設定 HTTP 目標，因此能更輕鬆地建立工作，且服務供應商可設定佇列中所有工作的目標 (例如，如果原始後端當機，可將流量轉送至其他後端)，因此能獲得更多控制權。

您可以在佇列層級設定下列項目：

- **標頭**：如果在佇列層級指定標頭，系統會為佇列中的所有工作插入/更新標頭。
- **HTTP 方法**：如果在佇列層級指定 HTTP 方法，系統會覆寫佇列中所有工作的 HTTP 方法。
- **目標 URI**：主機、路徑、查詢、通訊埠、配置 (HTTP 或 HTTPS) 可以個別覆寫。
- **授權**：在佇列層級指定的 OIDC/OAuth 設定會覆寫工作層級的 OIDC/OAuth 設定。

部署第二個 Cloud Run 服務

部署第二個 Cloud Run 服務，稍後將做為 HTTP URI 覆寫的目標：

```
SERVICE2=hello2
REGION=us-central1

gcloud run deploy $SERVICE2 \
  --allow-unauthenticated \
  --image=gcr.io/cloudrun/hello \
  --region=$REGION
```

儲存服務網址的主機，以供日後使用：

```
SERVICE2_URL=$(gcloud run services describe $SERVICE2 --region $REGION --format
'value(status.url)')
SERVICE2_HOST=$(echo $SERVICE2_URL | sed 's,http[s]*://,,g')
```

建立含有路由設定的 Cloud Tasks 佇列

建立佇列，並使用 HTTP URI 覆寫將路由設定至第二個 Cloud Run 服務。

```
QUEUE2=http-queue-uri-override

gcloud beta tasks queues create $QUEUE2 \
  --http-uri-override=host:$SERVICE2_HOST \
  --location=$LOCATION
```

請注意，URI 覆寫是指第二個 Cloud Run 服務。加入佇列的任何 HTTP 工作都會覆寫原始 URI 主機。您可以查看佇列設定：

```
gcloud beta tasks queues describe $QUEUE2 --location=$LOCATION
```

您應該會看到 `httpTarget` 有一個 `uriOverride` 指向第二個服務的主機：

```
httpTarget:
  uriOverride:
    host: hello2-idcwffc3yq-uc.a.run.app
    pathOverride: {}
    queryOverride: {}
...
```

暫時暫停佇列，以便觀察 HTTP 工作的建立過程：

```
gcloud tasks queues pause $QUEUE2 --location=$LOCATION
```

步驟 6 · 約 2 分鐘

6. 為佇列建立及測試 HTTP 工作，並設定轉送功能

在這個步驟中，您將建立 HTTP 工作，以第一個服務為目標，並觀察佇列如何覆寫其 URI，指向第二個服務。

建立 HTTP 工作

使用第一個服務的網址建立 HTTP 工作：

```
gcloud tasks create-http-task \  
  --queue=$QUEUE2 \  
  --location=$LOCATION \  
  --url=$SERVICE1_URL \  
  --method=GET
```

測試 HTTP 工作

繼續處理佇列：

```
gcloud tasks queues resume $QUEUE2 --location=$LOCATION
```

您應該會看到第二個 (而非第一個) Cloud Run 服務收到來自 Cloud Tasks 的 HTTP GET 要求，這是因為覆寫作業：

```
gcloud logging read "resource.type=cloud_run_revision AND  
resource.labels.service_name=$SERVICE2" --limit 1
```

```
---  
httpRequest:  
  latency: 0.228982142s  
  protocol: HTTP/1.1  
  remoteIp: 35.187.132.84  
  requestMethod: GET  
  requestSize: '426'  
  requestUrl: https://hello2-idcwffc3yq-uc.a.run.app/  
  responseSize: '5510'  
  serverIp: 216.239.34.53  
  status: 200  
  userAgent: Google-Cloud-Tasks
```

7. 使用轉送設定變更待處理工作

您也可以使用轉送設定，變更佇列中所有待處理工作的 HTTP URI。如果後端服務停止運作，且您想快速轉送至其他服務，這項功能就非常實用。讓我們看看這個步驟的運作方式。

再次暫停佇列：

```
gcloud tasks queues pause $QUEUE2 --location=$LOCATION
```

建立以 `google.com` 做為工作網址的 HTTP 工作：

```
gcloud tasks create-http-task \  
  --queue=$QUEUE2 \  
  --location=$LOCATION \  
  --url=https://www.google.com \  
  --method=GET
```

佇列已暫停，因此工作處於待處理狀態。

現在，請更新 HTTP URI 覆寫，將其指向第一個服務。這會將待處理工作的宿主從 `google.com` 覆寫為第一個服務的宿主：

```
SERVICE1_URL=$(gcloud run services describe $SERVICE1 --region $REGION --format  
'value(status.url)')  
SERVICE1_HOST=$(echo $SERVICE1_URL | sed 's,http[s]*://,,g')  
  
gcloud beta tasks queues update $QUEUE2 \  
  --http-uri-override=host:$SERVICE1_HOST \  
  --location=$LOCATION
```

繼續處理佇列：

```
gcloud tasks queues resume $QUEUE2 --location=$LOCATION
```

您應該會看到第一個 Cloud Run 服務收到來自 Cloud Tasks 的 HTTP GET 要求 (而非 `google.com`)，這是因為覆寫所致：

```
gcloud logging read "resource.type=cloud_run_revision AND  
resource.labels.service_name=$SERVICE1" --limit 1
```

httpRequest:

latency: 0.228982142s

protocol: HTTP/1.1

remoteIp: 35.187.132.84

requestMethod: GET

requestSize: '426'

requestUrl: https://hello1-idcwffc3yq-uc.a.run.app/

responseSize: '5510'

serverIp: 216.239.34.53

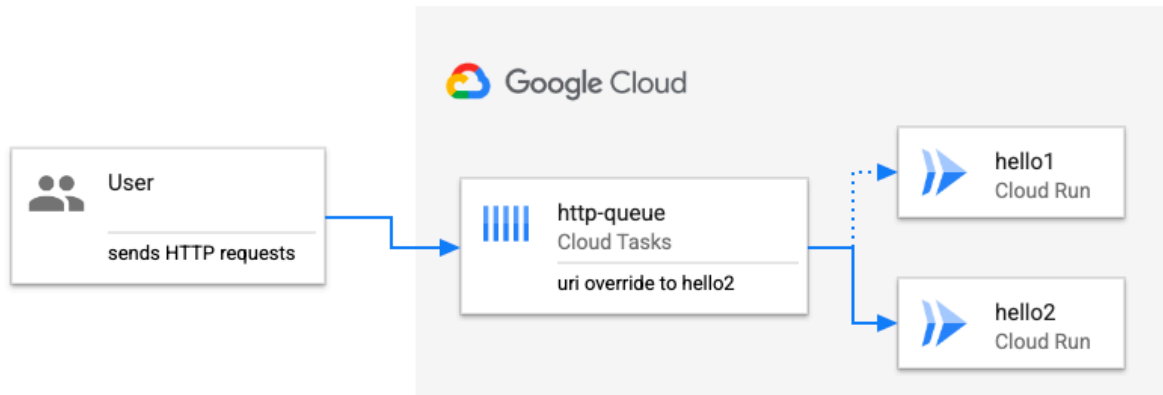
status: 200

userAgent: Google-Cloud-Tasks

8. 為 BufferTask API 建立佇列

一般來說，您會使用 Tasks API (透過 `gcloud` 或 Tasks 用戶端程式庫) 建立工作。這會造成應用程式負擔，必須使用用戶端程式庫將 HTTP 要求包裝成工作，也會在應用程式和 Tasks 用戶端程式庫之間建立依附元件。

在本步驟中，您會瞭解如何運用佇列層級的 HTTP URI 覆寫和新的 BufferTask API，只要傳送 HTTP 要求，就能更輕鬆地建立 HTTP 目標工作。現在，任何可傳送 HTTP 要求的應用程式都能建立 HTTP 目標工作。



什麼是 BufferTask API ?

CreateTask API 是建立工作項目的舊方法，需要用戶端將工作項目物件傳送至 API，並設定所有必填欄位。

BufferTask API 是一項新功能，可讓使用者建立 HTTP 工作，不必提供任何工作設定 (HTTP 網址、標頭、授權)，只要將訊息或要求主體傳送至 Buffer API 即可。

這樣一來，您就能更輕鬆地與服務整合，因為現在可以在服務前端部署 Cloud Tasks，不必在用戶端變更任何程式碼。傳送至 BufferTask API 的任意 HTTP 要求都會包裝成 Task 物件，並傳送至佇列層級設定的目的地。

如要使用 BufferTask API，佇列必須設定目標 URI 設定，換句話說，使用 BufferTask API 的前提是必須啟用佇列層級的路由設定功能。

建立含有路由設定的 Cloud Tasks 佇列

建立佇列，並設定指向上一步部署的第一個服務的路由設定：

```
SERVICE1=hello1
SERVICE1_URL=$(gcloud run services describe $SERVICE1 --region $REGION --format 'value(status.url)')
SERVICE1_HOST=$(echo $SERVICE1_URL | sed 's,http[s]*://,,g')
QUEUE3=http-queue-uri-override-buffer

gcloud beta tasks queues create $QUEUE3 \
  --http-uri-override=host:$SERVICE1_HOST \
  --location=$LOCATION
```

暫時暫停佇列，以便觀察 HTTP 工作的建立過程：

```
gcloud tasks queues pause $QUEUE3 --location=$LOCATION
```

9. 使用 BufferTask API 緩衝處理 HTTP 要求

在本步驟中，您將使用 BufferTask API 緩衝處理簡單的 HTTP GET 或 POST 要求。在幕後，Cloud Tasks 會將這些 HTTP 要求包裝成 HTTP 工作，並使用佇列的預設轉送設定。

首先，請登入以取得存取權杖，並設定部分變數：

```
gcloud auth application-default login
ACCESS_TOKEN=$(gcloud auth application-default print-access-token)
PROJECT_ID=$(gcloud config get-value project)
TASKS_QUEUES_API="https://cloudtasks.googleapis.com/v2beta3/projects/$PROJECT_ID/locations/$LOCATION/queues"
```

建立 HTTP 工作

使用 BufferTask API 建立 HTTP 工作。請注意，這是簡單的 HTTP GET 要求，不需要建立工作：

```
curl -X GET "$TASKS_QUEUES_API/$QUEUE3/tasks:buffer" \
-H "Authorization: Bearer $ACCESS_TOKEN"
```

使用 HTTP POST 和主體建立另一個 HTTP 工作：

```
curl -X POST "$TASKS_QUEUES_API/$QUEUE3/tasks:buffer" \
-H "Authorization: Bearer $ACCESS_TOKEN" \
-d '{"message': 'Hello World'}"
```

選用：您也可以使用用戶端程式庫建立 HTTP 工作。舉例來說，您可以查看 [Program.cs](#) 的 C# 範例，瞭解如何直接將 HTTP GET 要求傳送至 BufferTask API，不必將要求包裝在 `Task` 中，也不需要 Cloud Tasks 的用戶端程式庫：

```
var BufferTaskApiUrl =
$"https://cloudtasks.googleapis.com/v2beta3/projects/{ProjectId}/locations/{Location}/queues/{Queue}/tasks:buffer";

using (var client = new HttpClient())
{
    client.DefaultRequestHeaders.Add("Authorization", $"Bearer {AccessToken}");
    var response = await client.GetAsync(BufferTaskApiUrl);
    var content = await response.Content.ReadAsStringAsync();
    Console.WriteLine($"Response: {content}");
}
```

您可以按照下列方式執行：

```
dotnet run $PROJECT_ID $LOCATION $QUEUE3 $ACCESS_TOKEN
```

BufferTask API 會負責從 HTTP 要求建立工作，並從佇列的路由設定中新增 URI 的網址。

測試 HTTP 工作

繼續處理佇列：

```
gcloud tasks queues resume $QUEUE3 --location=$LOCATION
```

您應該會看到 Cloud Run 服務收到來自 Cloud Tasks 的 HTTP GET 和 POST 要求：

```
gcloud logging read "resource.type=cloud_run_revision AND  
resource.labels.service_name=$SERVICE1" --limit 4
```

```
---  
httpRequest:  
  latency: 0.002279292s  
  protocol: HTTP/1.1  
  remoteIp: 35.243.23.42  
  requestMethod: POST  
  requestSize: '777'  
  requestUrl: https://hello1-idcwffc3yq-uc.a.run.app/  
  responseSize: '5450'  
  serverIp: 216.239.32.53  
  status: 200  
  userAgent: Google-Cloud-Tasks  
...  
httpRequest:  
  latency: 0.228982142s  
  protocol: HTTP/1.1  
  remoteIp: 35.187.132.84  
  requestMethod: GET  
  requestSize: '426'  
  requestUrl: https://hello1-idcwffc3yq-uc.a.run.app/  
  responseSize: '5510'  
  serverIp: 216.239.34.53  
  status: 200  
  userAgent: Google-Cloud-Tasks
```

步驟 10 · 約 1 分鐘

10. 恭喜

恭喜，您已完成本程式碼研究室！

接著，您可以試用「[Cloud Tasks 做為 Pub/Sub 和 Cloud Run 之間的緩衝區](#)」，查看實際範例，瞭解 Cloud Tasks 的這些新功能如何協助您輕鬆建立服務之間的緩衝佇列。

清除 (選用)

為避免產生費用，建議您清除資源。

如果不需要專案，只要刪除即可：

```
gcloud projects delete $PROJECT_ID
```

如果需要專案，可以個別刪除資源。

刪除 Cloud Run 服務：

```
gcloud run services delete $SERVICE1 --region $REGION
gcloud run services delete $SERVICE2 --region $REGION
```

刪除 Cloud Tasks 佇列：

```
gcloud tasks queues delete $QUEUE1 --location=$LOCATION
gcloud tasks queues delete $QUEUE2 --location=$LOCATION
gcloud tasks queues delete $QUEUE3 --location=$LOCATION
```

涵蓋內容

- 如何建立 HTTP 目標工作。
 - 如何使用新的佇列層級 HTTP URI 覆寫功能，建立 HTTP 目標工作。
 - 如何使用新的佇列層級 HTTP URI 覆寫功能，變更待處理工作。
 - 如何使用新的 BufferTask API 更輕鬆地緩衝處理 HTTP 要求。
-